

FiaCraft Report



Battaglia Marco 0512116374
di Caprio Domenico 0512118543
Lavorato Giovanni 0512116371

<https://github.com/MarcoBattag/FiaCraft>

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Obiettivi	1
2	Tecnologie Utilizzate	2
2.1	Il Modello VPT (Video Pre-Training)	2
2.2	Caratterizzazione dell'Agente (PEAS)	3
2.3	Caratteristiche dell'Ambiente	4
2.4	Configurazione dell'Ambiente	5
3	Gestione dei Dati: Dal Dataset alla Pulizia	6
3.1	Acquisizione dei Dati (Download)	6
3.2	Filtraggio e Selezione: La funzione di scarto	6
3.3	Ottimizzazione dei contenuti: La funzione di taglio	8
3.4	Data Augmentation: Tecnica di Mirroring	9
3.5	Validazione e Debug: Visualizzazione dei Movimenti	10
4	Sfide Tecniche e Soluzioni Proposte	11
4.1	1. Orizzonte Temporale e Gerarchia delle Azioni	11
4.2	2. Il Problema della Rappresentazione Visiva	11
4.3	3. Sincronizzazione Input-Output e Latenza	11
5	Il Processo di Behavioural Cloning: Imitazione e Stabilità	12
5.1	Architettura della Loss Function	12
5.2	Ottimizzazione e Data Augmentation	13
6	Reinforcement Learning: Ottimizzazione e Reward Shaping	14
6.1	L'algoritmo di Ottimizzazione	14
6.2	Reward Shaping Gerarchico e Dinamico	14
6.3	Ciclo di Addestramento e Checkpointing	15
6.4	Analisi Avanzata della Strategia RL	16
6.5	Conclusioni sulla fase di Reinforcement Learning	16
7	Testing e Validazione	17
7.1	Analisi della Training Loss	17
7.2	Ambiente di Valutazione e Analisi dei Risultati	17

1 Introduzione

FiaCraft è un progetto universitario volto ad addestrare agenti su Minecraft utilizzando MineRL e l'apprendimento per imitazione.

Il nostro team ha lavorato attivamente all'estensione della libreria originale, introducendo nuove funzionalità per migliorare l'addestramento e la versatilità degli agenti:

- **Efficienza computazionale:** Ottimizzazione dei processi tramite l'integrazione e il supporto completo alla **GPU**.
- **Gestione dei Dati:** Raffinamento delle procedure di raccolta per un apprendimento più rapido e preciso.
- **Logiche di Reward:** Implementazione di sistemi di ricompensa personalizzati per task complessi, come il crafting del piccone in legno.

1.1 Obiettivi

L'obiettivo del nostro progetto è creare un agente che sappia cavarcela su Minecraft, partendo da zero. In pratica, vogliamo che l'agente impari a fare le stesse cose che fa un giocatore umano appena entra in un nuovo mondo.

Nello specifico, l'agente deve riuscire a:

- Trovare gli alberi e raccogliere il legno;
- Trasformare il legno in assi e stick;
- Creare un banco da lavoro;
- Raggiungere l'obiettivo finale: costruire un **piccone di legno**.

Questi sono i passaggi base per iniziare qualsiasi partita. Per riuscirci, abbiamo usato il sistema della clonazione comportamentale, facendogli vedere come fanno gli umani, e abbiamo aggiunto dei reward ogni volta che l'agente colleziona l'oggetto giusto, così da spingerlo a finire tutto il lavoro fino al piccone.

2 Tecnologie Utilizzate

In questa sezione spieghiamo quali strumenti abbiamo scelto per far funzionare l'agente e come interagiscono tra loro.

- **Minecraft e MineRL:** Sono la base del progetto. **MineRL** è una libreria che permette al codice Python di comunicare con Minecraft, controllando l'agente e leggendo i dati di gioco.
- **OpenAI VPT (Video Pre-Training):** Abbiamo usato un modello pre-addestrato da OpenAI su migliaia di ore di video di giocatori umani. Questo fornisce all'agente una conoscenza di base su come muoversi e interagire.
- **PyTorch:** È la libreria che gestisce la rete neurale.
- **Python:** Il linguaggio principale che collega tutti i componenti e ci ha permesso di scrivere le logiche di reward e gestire l'addestramento.

2.1 Il Modello VPT (Video Pre-Training)

Per il "cervello" dell'agente abbiamo scelto di utilizzare **VPT**, un modello di rete neurale profonda sviluppato da OpenAI. La particolarità di questo modello è che è stato addestrato su un dataset immenso di video di giocatori umani, permettendo all'agente di imparare non solo a muoversi, ma anche a interagire con l'interfaccia di Minecraft.

L'architettura del modello si basa su tre pilastri fondamentali che sono stati essenziali per il nostro progetto:

- **Percezione Visiva (CNN):** Attraverso strati convoluzionali, il modello elabora le immagini del gioco. Questo permette all'agente di "vedere" e riconoscere visivamente gli alberi da abbattere o la Crafting Table una volta piazzata.
- **Modellazione Temporale (Transformer):** L'uso di blocchi Transformer permette all'agente di avere una "memoria a breve termine". Questo è fondamentale per il crafting: l'agente deve ricordare di aver appena creato delle assi per poter poi procedere a costruire i stick o il banco da lavoro.

- **Azioni Complesse:** VPT gestisce sia azioni discrete (come i tasti di movimento) sia azioni continue (come il movimento della visuale), rendendo il comportamento dell'agente molto fluido e simile a quello umano.

Abbiamo sfruttato questo modello pre-addestrato come base, concentrando il nostro lavoro sulla definizione dei **pesi** e delle **ricompense** necessari a guidare queste capacità verso l'obiettivo finale: la fabbricazione del piccone.

2.2 Caratterizzazione dell'Agente (PEAS)

Per descrivere formalmente l'ambiente in cui opera il nostro agente e le sue capacità, utilizziamo il framework **PEAS**:

Performance (Prestazioni): Il successo dell'agente viene misurato in base alla capacità di completare la sequenza di crafting. I criteri di valutazione includono:

- Raccolta di tronchi di legno dagli alberi;
- Produzione di assi e stick;
- Creazione del banco da lavoro (*Crafting Table*);
- Fabbricazione del **piccone di legno** (obiettivo massimo);
- Efficienza nel completare i task nel minor tempo possibile.

Environment (Ambiente): Il mondo di **Minecraft**. Si tratta di un ambiente 3D, dinamico, stocastico e parzialmente osservabile, composto da blocchi con cui l'agente può interagire

Actuators (Attuatori): I mezzi con cui l'agente agisce nel mondo. Includono i comandi della tastiera e il movimento del mouse per orientare la visuale e interagire con i blocchi .

Sensors (Sensori): Gli strumenti con cui l'agente riceve informazioni. Il sensore principale è la **visuale in prima persona**, accompagnata dai dati dell'inventario che permettono di sapere quanti oggetti sono stati raccolti o creati.

2.3 Caratteristiche dell'Ambiente

L'ambiente di Minecraft, osservato dal punto di vista dell'agente, presenta delle sfide specifiche che abbiamo dovuto considerare durante l'addestramento:

- **Parzialmente osservabile:** L'agente non vede tutto il mondo, ma solo quello che ha davanti agli occhi. Se un albero è alle sue spalle, deve girarsi per "scoprirlo".
- **Stocastico:** Ogni volta che inizia una partita, la posizione degli alberi e la conformazione del terreno cambiano. Non esiste un percorso fisso pre-programmato.
- **Sequenziale:** L'ordine delle azioni è tutto. Non si può creare un piccone senza aver prima creato il banco da lavoro, e non si può fare il banco senza la legna.
- **Dinamico:** Il mondo cambia mentre l'agente agisce.
- **Misto (Discreto/Continuo):** L'agente deve gestire tasti "on/off" (come saltare) e movimenti fluidi e precisi (come mirare un blocco con il mouse).
- **Singolo Agente:** Nel nostro progetto, l'agente è l'unico protagonista e non deve competere o collaborare con altri giocatori.

2.4 Configurazione dell'Ambiente

In questa sezione descriviamo come abbiamo preparato il sistema per far girare Minecraft e l'addestramento dell'agente. La sfida principale è stata far comunicare correttamente il simulatore di gioco con gli algoritmi di intelligenza artificiale.

- **Integrazione GPU:** Per gestire la complessità del modello VPT, abbiamo configurato l'ambiente tramite **CUDA**. Questo ha permesso di spostare il calcolo sulla scheda video, velocizzando drasticamente i tempi di risposta dell'agente.
- **Software e Librerie:** Abbiamo utilizzato Python come linguaggio principale, configurando un ambiente virtuale con tutte le dipendenze necessarie.
- **Gestione dei Timeout:** Una parte importante della configurazione ha riguardato la sincronizzazione tra lo script Python e il gioco, evitando che Minecraft si chiudesse o si bloccasse durante le fasi di elaborazione più intense.

3 Gestione dei Dati: Dal Dataset alla Pulizia

Il cuore dell'addestramento del nostro agente risiede nel modo in cui abbiamo gestito e preparato i dati. Non si è trattato solo di scaricare dei file, ma di un vero e proprio percorso di elaborazione per trasformare ore di gameplay in informazioni utili all'apprendimento.

3.1 Acquisizione dei Dati (Download)

Il viaggio è iniziato con l'acquisizione del dataset di OpenAI dedicato a VPT. Questo processo ha comportato:

- L'utilizzo di script specifici per il download automatico di file video (.mp4) e dei relativi file .json, che contengono le azioni dei giocatori.
- La gestione di una mole di dati considerevole, che ha richiesto un'organizzazione strutturata delle cartelle per separare i vari "episodi" di gioco.

3.2 Filtraggio e Selezione: La funzione di scarto

Non tutti i video del dataset OpenAI erano utili: molti mostravano giocatori in fasi avanzate del gioco con inventari già pieni. Abbiamo quindi sviluppato una **funzione di scarto** per isolare solo i "nuovi inizi".

Il criterio è semplice: se l'inventario all'inizio del video non è vuoto, il video viene scartato. Questo garantisce che l'agente impari a giocare partendo dalle stesse condizioni iniziali in cui si troverà durante i test.

```

1 def is_inventory_empty(jsonl_path):
2     """
3     Controlla se il primo frame del file JSONL ha un inventario
4     vuoto.
5     """
6     try:
7         with open(jsonl_path, "r", encoding="utf-8", errors="ignore") as file:
8             first_line = file.readline()
9             if not first_line:
10                 return False
11
12             data = json.loads(first_line)
13             inventory = data.get("inventory", [])
14             # L'agente deve iniziare la sessione senza alcun
15             oggetto
16             return len(inventory) == 0
17     except Exception as e:
18         print(f"Errore nel leggere {jsonl_path}: {e}")
19         return False

```

Listing 1: Script per il filtraggio degli episodi basato sull'inventario iniziale

Grazie a questa selezione, siamo riusciti a ottenere un dataset, composto esclusivamente da sessioni in cui il giocatore umano parte dalla raccolta manuale della legna, esattamente come dovrà fare il nostro agente durante i test.

3.3 Ottimizzazione dei contenuti: La funzione di taglio

Spesso i video contengono tempi morti o azioni irrilevanti dopo il raggiungimento dell'obiettivo. La nostra **funzione di taglio** interviene per "potare" i dati, mantenendo solo i segmenti dove avviene il progresso effettivo verso il piccone.

Questo script analizza la timeline dell'inventario e taglia il video non appena l'obiettivo prefissato viene raggiunto, evitando di confondere l'agente con dati post-obiettivo.

```
1 def get_cut_timestamp(jsonl_path):
2     """
3     Individua il tick in cui il piccone viene creato.
4     Scarta il video se l'oggetto e gia presente all'inizio.
5     """
6     # ... caricamento linee JSONL ...
7     for line in lines:
8         data = json.loads(line)
9         inventory = data.get("inventory", [])
10
11        # Controllo iniziale: scarta se il piccone e gia presente
12        if first_inventory_check and has_wooden_pickaxe(inventory):
13            return None, None, False
14
15        # Individuazione momento del crafting
16        if has_wooden_pickaxe(inventory):
17            cut_tick = data["tick"]
18            cut_time = (data["milli"] - first_milli) / 1000.0
19            return cut_tick, cut_time, True
20    return None, None, False
21
22 def trim_video(video_path, output_path, cut_time):
23     # Utilizza OpenCV per tagliare il video al cut_frame calcolato
24     # aggiungendo un buffer di 1 secondo per visualizzare l'
25     # obiettivo
26     cut_frame = int(cut_time * fps) + int(fps * 1.0)
27     # ... scrittura del nuovo file video ...
```

Listing 2: Script per il taglio sincronizzato di video e metadati

Grazie a questa procedura, abbiamo ridotto drasticamente il rumore nel dataset, eliminando ore di gameplay irrilevanti e riducendo lo spazio occupato su disco, velocizzando così le epoche di addestramento successive.

3.4 Data Augmentation: Tecnica di Mirroring

Per aumentare la quantità di dati a disposizione e migliorare la capacità di generalizzazione dell'agente, abbiamo implementato una tecnica di **Data Augmentation** tramite il **mirroring**.

Questa procedura ci ha permesso di raddoppiare il dataset effettivo senza dover scaricare nuovi video, applicando le seguenti trasformazioni:

- **Trasformazione Visiva:** Ogni frame del video è stato riflesso orizzontalmente utilizzando la libreria OpenCV.
- **Trasformazione delle Azioni:** È stato fondamentale specchiare anche i metadati delle azioni. Ad esempio, se nel video originale il giocatore premeva il tasto per girare a destra, nella versione specchiata l'azione è stata convertita in una rotazione verso sinistra.
- **Riduzione del Bias:** Questa tecnica impedisce all'agente di imparare percorsi mnemonici (es. "c'è sempre un albero a destra"), costringendolo a riconoscere gli oggetti indipendentemente dalla loro posizione nel campo visivo.

```
1 # Inversione dei tasti di movimento
2 if key == "key.keyboard.a":
3     new_keys.append("key.keyboard.d")
4 elif key == "key.keyboard.d":
5     new_keys.append("key.keyboard.a")
6
7 # Inversione movimento mouse e specchiatura video
8 if not data.get("isGuiOpen", False):
9     data["mouse"]["dx"] = -data["mouse"]["dx"]
10    mirrored_frame = cv2.flip(frame, 1)
11 else:
12    mirrored_frame = frame # Non specchiare se la GUI è aperta
```

Listing 3: Logica di inversione input e mirroring condizionale

Grazie al mirroring, l'agente ha potuto osservare il doppio delle situazioni di gioco, diventando molto più flessibile e pronto a reagire correttamente a conformazioni del terreno mai viste prima.

3.5 Validazione e Debug: Visualizzazione dei Movimenti

Per garantire che il processo di *mirroring* e il filtraggio dei dati fossero avvenuti correttamente, abbiamo sviluppato uno strumento di visualizzazione personalizzato utilizzando la libreria **Pygame**.

Questo script permette di "riprodurre" i movimenti del mouse estratti dai file JSONL su una finestra grafica. Questo passaggio di validazione è stato essenziale per:

- **Verifica del Mirroring:** Confermare visivamente che l'inversione delle coordinate x e y corrispondesse effettivamente alla riflessione del video.
- **Analisi del Comportamento Umano:** Osservare le traiettorie seguite dai giocatori durante il puntamento dei blocchi (es. alberi), aiutandoci a capire meglio la distribuzione dei dati di input.
- **Debug Tecnico:** Identificare eventuali anomalie nelle coordinate (es. picchi improvvisi o dati fuori scala) prima di darli in pasto alla rete neurale.

```
1 def main(file_path):
2     # Estrazione coordinate x, y dal file JSONL
3     coordinates = extract_coordinates(file_path)
4     # Inizializzazione Pygame per il rendering grafico
5     pygame.init()
6     screen = pygame.display.set_mode((WIDTH, HEIGHT))
7
8     while running:
9         # Pulisci lo schermo e disegna la posizione del mouse
10        screen.fill(BACKGROUND_COLOR)
11        if index < len(coordinates):
12            x, y = coordinates[index]
13            # Disegno di un indicatore circolare sulla posizione
14            # corrente
15            pygame.draw.circle(screen, BALL_COLOR, (x, y),
16                               BALL_RADIUS)
17            index += 1
18            # Aggiornamento dello schermo a 30 FPS (sincronizzato con
19            # il video)
20            pygame.display.flip()
21            clock.tick(FPS)
```

Listing 4: Script di visualizzazione Pygame per il debug del movimento mouse

4 Sfide Tecniche e Soluzioni Proposte

Lo sviluppo di un agente intelligente in Minecraft presenta ostacoli unici che richiedono particolari tecniche per poter essere affrontate. Di seguito analizziamo le criticità affrontate e le strategie adottate per superarle.

4.1 1. Orizzonte Temporale e Gerarchia delle Azioni

Problema: Per creare un piccone, l'agente deve compiere centinaia di azioni elementari (mirare, colpire, aprire menu). Un singolo errore all'inizio della catena compromette l'intero risultato. **Soluzione:** Abbiamo utilizzato il **Behavioural Cloning** su un dataset filtrato. Invece di far esplorare il mondo a caso, l'agente impara direttamente le traiettorie ottimali dai dati umani, riducendo drasticamente lo spazio di ricerca delle azioni.

4.2 2. Il Problema della Rappresentazione Visiva

Problema: Minecraft ha un mondo dinamico con illuminazione variabile, biomi diversi e meteo cangiante. L'agente rischia di confondersi se vede un albero in un deserto rispetto a uno in una foresta. **Soluzione:** Abbiamo applicato tecniche di **Data Augmentation (Mirroring)**. Specchiando i video e i relativi input (mouse e tastiera), abbiamo raddoppiato la varietà di situazioni visive, insegnando al modello che la logica del crafting rimane invariata indipendentemente dall'orientamento spaziale.

4.3 3. Sincronizzazione Input-Output e Latenza

Problema: La latenza tra l'osservazione del frame e l'esecuzione del comando può portare l'agente a "mancare" i blocchi. **Soluzione:** Lo script di **Visualizzazione e Debug** sviluppato con Pygame ci ha permesso di verificare la perfetta sincronizzazione tra le coordinate del mouse estratte dai file JSONL e l'azione visiva nel video. Questo ha garantito che il dataset di addestramento fosse privo di sfasamenti temporali dannosi.

5 Il Processo di Behavioural Cloning: Imitazione e Stabilità

La fase centrale del progetto consiste nell'addestramento del modello tramite **Behavioural Cloning**. In questa fase, l'agente non impara per tentativi ed errori, ma cerca di massimizzare la probabilità di eseguire la stessa azione scelta da un umano in una determinata situazione visiva.

5.1 Architettura della Loss Function

Per garantire che l'agente impari efficacemente senza dimenticare le basi motorie del modello pre-addestrato (VPT), abbiamo implementato una funzione di perdita composta:

$$Loss = NLL + \lambda \cdot D_{KL}$$

- **Negative Log-Likelihood (NLL):** Misura quanto bene l'agente sta imitando l'azione umana. Minimizzare la NLL equivale a massimizzare la probabilità dell'azione corretta.
- **Divergenza di Kullback-Leibler (KL):** Agisce come un termine di regolarizzazione. Impedisce al modello di allontanarsi eccessivamente dalla "policy originale" di OpenAI, mantenendo l'agente coerente e fluido nei movimenti di base.

```
1 # Calcolo probabilita dell'azione del modello corrente
2 pi_distribution, _, new_agent_state = policy.
  get_output_for_observation(agent_obs, agent_state, first_tensor
  )
3
4 # Calcolo probabilita dell'azione del modello originale (Frozen)
5 with th.no_grad():
6     original_pi_distribution, _, _ = original_policy.
      get_output_for_observation(agent_obs, agent_state, first_tensor
      )
7
8 # Calcolo delle componenti della Loss
9 log_prob = policy.get_logprob_of_action(pi_distribution,
    agent_action)
10 kl_div = policy.get_kl_of_action_dists(pi_distribution,
    original_pi_distribution)
11
12 current_nll = -log_prob
13 step_loss = current_nll + KL_LOSS_WEIGHT * kl_div
```

Listing 5: Loop di calcolo Loss con regolarizzazione KL

5.2 Ottimizzazione e Data Augmentation

Per rendere l'addestramento più robusto, abbiamo introdotto diverse tecniche avanzate direttamente nel loop di training:

- **Visual Augmentation:** Utilizzando `torchvision.transforms`, ogni frame viene alterato casualmente in luminosità, contrasto e saturazione (*ColorJitter*). Questo impedisce all'agente di memorizzare specifici colori e lo costringe a riconoscere le forme (es. alberi) in diverse condizioni di luce.
- **Memoria Temporale (Hidden States):** Essendo il modello basato su Transformer e LSTM, il codice gestisce meticolosamente gli *hidden states*. Lo stato dell'agente viene preservato lungo le sequenze (`seq_len=32`) e staccato dal grafo computazionale (`detach`) solo al termine della sequenza per ottimizzare la memoria VRAM.
- **Gradient Clipping:** Abbiamo impostato un `max_grad_norm` di 5.0 per evitare il problema dell'esplosione del gradiente, tipico delle reti ricorrenti profonde.
- **Cosine Annealing Scheduler:** Il *learning rate* non è fisso, ma segue una curva cosinusoidale che decresce nel tempo, permettendo al modello di stabilizzarsi con precisione nelle fasi finali dell'addestramento.

6 Reinforcement Learning: Ottimizzazione e Reward Shaping

Dopo aver stabilizzato il comportamento dell'agente con il *Behavioural Cloning*, abbiamo implementato una fase di **Reinforcement Learning**. In questa fase, l'agente interagisce direttamente con l'ambiente Minecraft e aggiorna la propria *policy* per massimizzare una funzione di ricompensa specifica, passando da un modello puramente imitativo a uno orientato all'obiettivo (*goal-oriented*).

6.1 L'algoritmo di Ottimizzazione

Per questa fase abbiamo utilizzato un approccio basato su **Policy Gradient**, ottimizzando la rete tramite l'algoritmo **RMSprop** con un *learning rate* di 0.0001. Il calcolo della perdita (*loss*) avviene ogni 64 step o alla conclusione di un task, utilizzando il logaritmo della probabilità dell'azione intrapresa pesata per il vantaggio ottenuto:

$$Loss = -E[\log \pi(a_t|s_t) \cdot \hat{A}_t]$$

Dove $\hat{A}_t$ rappresenta il vantaggio normalizzato della ricompensa ottenuta. Questo spinge l'agente a rendere più probabili le azioni che portano a premi elevati.

6.2 Reward Shaping Gerarchico e Dinamico

Il cuore dell'addestramento RL risiede nella funzione `compute_reward_and_stop`. Abbiamo progettato un sistema di ricompense **gerarchico** per riflettere l'albero tecnologico di Minecraft:

- **Ricompense di Progresso:** Ogni materiale chiave ha un valore crescente: *Log* (2.0), *Planks* (4.0), *Stick* (6.0), *Crafting Table* (15.0) fino al *Wooden Pickaxe* (500.0).
- **Gestione del "Rendimento Decrescente":** Per evitare che l'agente accumuli solo legna senza progredire, la ricompensa per i tronchi viene drasticamente ridotta (moltiplicatore 0.1) se l'agente ha già prodotto assi, spingendolo verso il task successivo.
- **Ricompense per Eventi Critici:** Abbiamo inserito un premio speciale di +50.0 quando l'agente **piazza** una *Crafting*

Table. Questo è un momento critico: il codice rileva la diminuzione dell'oggetto nell'inventario per dedurre l'azione di posizionamento nel mondo.

- **Penalità per Materiali Inutili:** Per focalizzare l'agente, abbiamo introdotto piccole penalità per la raccolta di blocchi non necessari al task, come terra o sabbia (-0.2).

6.3 Ciclo di Addestramento e Checkpointing

L'addestramento è strutturato su episodi. Per garantire la massima efficienza, l'intero loop di calcolo (osservazioni, azioni e gradienti) è gestito su **GPU CUDA**. Il sistema salva automaticamente i pesi del modello (`best_wooden_pickaxe.weights`) ogni volta che l'agente stabilisce un nuovo record di punteggio cumulativo, permettendo di recuperare la versione più performante dell'agente al termine delle 25 epoche previste.

```
1 # Calcolo probabilit  e vantaggio
2 pi_logits, _, _ = agent.policy.get_output_for_observation(agent_obs
   , agent.hidden_state, ...)
3 log_prob = agent.policy.get_logprob_of_action(pi_logits,
   agent_action)
4
5 # Calcolo Loss pesata sull'Advantage normalizzato
6 adv_t = normalize(th.stack(batch_advantages).to(device))
7 loss = -(probs_t * adv_t.detach()).mean()
8
9 # Backpropagation
10 optimizer.zero_grad()
11 loss.backward()
12 optimizer.step()
```

Listing 6: Logica di aggiornamento della policy su GPU

6.4 Analisi Avanzata della Strategia RL

Oltre alla struttura di base del Reinforcement Learning, l'implementazione del progetto include diverse soluzioni tecniche per ottimizzare la convergenza del modello:

- **Gestione dello Stato Ricorrente:** Minecraft è un ambiente a *osservabilità parziale*: l'agente non vede cosa succede dietro di sé. Il codice preserva l' `agent.hidden_state` tra i vari step. Questo permette alla memoria interna della rete di mantenere informazioni critiche (es. "dove ho piazzato la crafting table?") anche quando l'oggetto esce dal campo visivo attuale.
- **Stabilità del Gradiente tramite Normalizzazione:** L'uso della funzione `normalize(tensor)` per le ricompense accumulate è fondamentale. Senza di essa, i picchi di reward (come i +500 del piccone) causerebbero aggiornamenti dei pesi troppo violenti, portando al collasso della rete. La normalizzazione riporta il segnale in un range gestibile, mantenendo l'apprendimento fluido.
- **Il Concetto di "High Water Mark" Dinamico:** La logica implementata non premia il possesso di un oggetto, ma l'evento del suo **incremento**. Questo trasforma il problema da una valutazione di stato a una valutazione di azione. L'introduzione di bonus extra per il "primo ottenimento" (es. +10 per il primo stick) funge da *curiosity-driven reward*, spingendo l'agente a esplorare nuove ricette di crafting.

```
1 bonus = 10.0 if current_max == 0 else 0.0
2 gain = float(diff * base_reward + bonus)
```

Listing 7: Dettaglio della logica di bonus per scoperta

6.5 Conclusioni sulla fase di Reinforcement Learning

In conclusione, l'integrazione del **Reinforcement Learning** ha rappresentato il punto di svolta per l'autonomia dell'agente. Mentre il Behavioural Cloning ha fornito la conoscenza dei movimenti umani, il processo di RL tramite PPO ha permesso di raffinare tale conoscenza, trasformandola in una strategia orientata all'efficienza.

7 Testing e Validazione

La fase di testing è cruciale per verificare se l'agente ha effettivamente appreso le dinamiche di gioco e se è in grado di generalizzare le conoscenze acquisite in nuovi ambienti generati proceduralmente. Durante questa fase, abbiamo monitorato non solo il raggiungimento degli obiettivi (il crafting), ma anche le metriche interne della rete neurale per assicurare la stabilità dell'apprendimento.

7.1 Analisi della Training Loss

Uno degli indicatori principali della salute del processo di addestramento è l'andamento della funzione di costo (*Training Loss*). Questo valore rappresenta numericamente la discrepanza tra l'azione predetta dall'agente e l'azione effettiva compiuta dal giocatore umano nel dataset di riferimento.

Come si può osservare dal grafico in Figura 1, la curva di apprendimento mostra un comportamento ottimale suddiviso in due fasi distinte:

1. **Discesa Rapida (Fast Learning):** Nei primi 500-600 secondi (corrispondenti ai primi 50 batch), la loss crolla drasticamente da un valore iniziale di circa 266 a circa 100. Questo indica che la rete neurale sta apprendendo molto velocemente le associazioni fondamentali "visione-azione".
2. **Convergenza e Stabilizzazione:** Dopo il primo periodo, la curva rallenta la discesa e si stabilizza attorno a un valore medio di 70-75. Le oscillazioni visibili sono fisiologiche nel Deep Learning e sono dovute alla variabilità dei mini-batch. Il fatto che la curva non risalga indica che non c'è stato *overfitting* e che il modello ha raggiunto un punto di equilibrio stabile.

7.2 Ambiente di Valutazione e Analisi dei Risultati

Per quantificare le capacità dell'agente, abbiamo condotto una sessione di testing su un campione di **50 episodi** distinti. Ogni episodio ha avuto una durata massima di **1500 passi** (step), dove ogni passo corrisponde a una singola azione decisionale della rete neurale (movimento, click, rotazione della visuale).

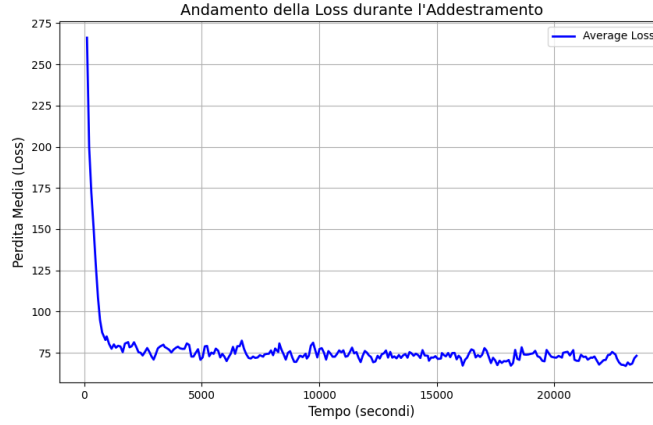


Figura 1: Andamento della Loss media nel tempo. Si nota la rapida convergenza iniziale e la successiva stabilizzazione, indice di un apprendimento efficace.

I risultati hanno evidenziato una netta distinzione nelle probabilità di successo tra i task di raccolta e quelli di crafting:

- Nell'**90%** dei casi, l'agente è riuscito a identificare, raggiungere e raccogliere correttamente i tronchi di legno (*logs*). Questo conferma l'efficacia del modello visivo nel riconoscere le risorse naturali.
- La percentuale scende al **10%** per la creazione delle assi di legno (*planks*), evidenziando la prima barriera significativa legata all'apertura e gestione dell'inventario.
- Solo nell'**1%** degli episodi l'agente è riuscito a completare la costruzione della *Crafting Table*.
- Mentre nell'**3%** l'agente riesce a creare gli stick o tramite l'utilizzo della crafting table o tramite l'utilizzo del crafting del suo inventario.
- Infine, l'obiettivo ultimo del **Piccone di Legno** è stato raggiunto nello **0%** dei casi.

Questi dati suggeriscono che, mentre la navigazione e l'interazione con il mondo fisico sono state apprese solidamente, la manipolazione sequenziale dell'interfaccia utente (GUI) per il crafting rappresenta il collo di bottiglia principale per le prestazioni attuali.

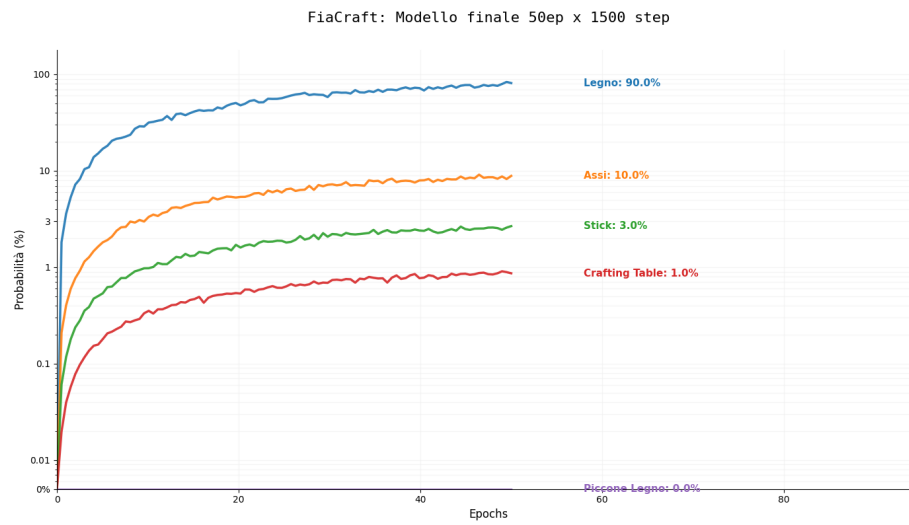


Figura 2: Istogramma dei tassi di successo sui 50 episodi di test. La scala logaritmica evidenzia il drastico calo di prestazioni man mano che la complessità del crafting aumenta.